

Conversational Planning Agent Design — v3.30

Nature of this document: Cross-domain methodology paper spanning LLM / NLU / IE-OR / UX, describing erpilot v3.30's **PlanningAgent** — wrapping **all** v3.25.9 → v3.29 IE/OR algorithms as **LLM-callable tools** so SMB owners can use world-class planning methods **just by talking**.

■ Prerequisites: design docs of all sprints v3.25.9 → v3.29 (algorithm foundation)

Abstract

The past 5 sprints (v3.25.9 → v3.29) completed **operations-research-grade IE/OR algorithms** — BOM multi-level, MRP-II, CLSP, TOC trilogy, demand forecasting — citing 50+ academic papers. However, **these algorithms exist only as Python services** callable by engineers, violating erpilot's North Star promise: *"natural language replaces training."* v3.30 closes this last-mile gap by wrapping 10 critical algorithms as `@register_tool`-registered LLM tools, enabling owners to say:

"Where's our bottleneck?" / "Should we take this order?" / "How many M6 bolts do we need next month?" / "Why is next week so busy?" / "What should I watch today?"

Plus the **Daily Briefing** killer feature aggregates **all upstream analysis** into 3-5 prioritized "things to watch today." All hard-writes use **ConfirmCard**; read tools return a `{summary, raw, warning}` three-segment structure for both human-friendly text and verifiable raw data. 15 structural-invariant tests pass.

Keywords: Conversational ERP, LLM tool wrapping, ConfirmCard, Daily Briefing, SMB UX, planning agent

1. Introduction: The Black-Box Problem of v3.25.9-v3.29

1.1 Fatal Contradiction

5 sprints' achievements:

Sprint	Academic achievement	Customer access
v3.25.9	BOM recursive explosion	❌ Read API only
v3.25.10	MRP-II + Wagner-Whitin $O(T^2)$	❌ Python service only
v3.26	Dixon-Silver CLSP heuristic	❌ Python service only
v3.27	Provenance + TOC + Counterfactual	❌ Python service only
v3.28	TA + DBR + Order Acceptance	❌ Python service only
v3.29	5-method auto-selection + MASE	❌ Python service only

Every ❌ means: SMB owners **cannot ask the AI** to use these features — directly violating erpilot's v3.0 strategic pivot's principle "**UI vs conversation → conversation.**"

1.2 Core Thesis

Algorithm value = quality × accessibility:

```
value = quality × accessibility
      = 95% × 0%      (after v3.25.9-v3.29 ship)
      = 0%
```

Even the most perfect Wagner-Whitin DP has zero value if SMB owners can't invoke it.

```
v3.30 → fixed
value = 95% × 90%
      = 85.5%
```

10 LLM-tool wrappers + Daily Briefing = value unlock of the entire IE/OR investment.

2. Methodology

2.1 LLM Tool Wrapper Pattern

Each algorithm wrapped using the same template:

```
@register_tool(
    name="<verb>_<noun>_tool",
    domain="planning",
    risk_tier=RiskTier.READ | HARD_WRITE,
    description="""<natural language description with example phrases>""",
    slots=[Slot(<arg>, <type>, required=<bool>, description=...)],
    required_permission="<rbac.code>",
)

async def _<impl>(db, user, <args>) -> Dict[str, Any]:
    # 1. Look up entities (product / part / WC)
    # 2. Call underlying service (already completed in prior sprints)
    # 3. Translate to "human-readable" summary + retain raw data + add warning
    return {
        "summary": "...",          # LLM directly renders this to user
        "raw": {...},              # programmatic access
        "warning": "⚠️ ...",       # legal / algorithmic limitation note
    }
```

2.2 Three-Segment Response Rationale

`{summary, raw, warning}` mirrors XAI's three principles:

Segment	Principle	Reference
<code>summary</code>	Comprehensibility — end-user can read	Doshi-Velez & Kim 2017
<code>raw</code>	Fidelity — no information loss, programmatically accessible	Lundberg-Lee 2017 SHAP
<code>warning</code>	Calibrated confidence — explicit limitations	Ribeiro et al. 2016 LIME

2.3 ConfirmCard Integration (Hard-write)

The only hard-write tool in v3.30: `commit_forecast_to_mps_with_confirm`. Flow:

```
User speaks → LLM calls tool
      ↓
  build ConfirmCard (summary + slots)
      ↓
ConfirmCard shown to owner for review
      ↓
Owner clicks "✅ Confirm"
      ↓
execute closure (real DB write)
```

2.4 Daily Briefing — Killer Feature

Owner enters office 9 AM, first question typically: "What should I watch today?"

`daily_briefing_tool` aggregates multi-algorithm output, producing prioritized 3-5 items:

```
Priority 1 (🔴): Low-stock parts (below min_stock)
Priority 2 (📄): SOs created in last 7 days
Priority 3 (📦): Draft POs not yet sent
Priority 4 (🏢): Latest MRP master + "check bottleneck" hint
Priority 99 (🌤️): Calm-day fallback
```

Design considerations:

- No overload: top 5 only (Miller 1956 cognitive limit 7 ± 2)
- Clickable depth: each item suggests next tool ("ask me 'where's the bottleneck'")
- Weather-style summary: "🌤️" instead of blank when calm — psychological reassurance

3. Implementation

```
backend/app/agents/domains/planning_llm_tools.py (~700 lines)
|
├─ Tool 1: forecast_demand_for_part      "Forecast next quarter's M6 bolt demand"
├─ Tool 2: commit_forecast_to_mps_with_confirm "Write forecast to MPS" ← only hard-write
├─ Tool 3: explain_planned_order_tool     "Why so many M6 bolts next week?"
├─ Tool 4: identify_bottlenecks_tool      "Where's our bottleneck?"
├─ Tool 5: counterfactual_capacity_tool    "What if we add 20% press capacity?"
├─ Tool 6: evaluate_order_acceptance_tool  "Should we take this order?"
├─ Tool 7: explore_pricing_curve_tool      "What's the lowest price we can accept?"
├─ Tool 8: compute_dbr_schedule_tool       "Plan production rhythm for this press"
├─ Tool 9: where_used_tool                 "Which products use the M6 bolt?"
└─ Tool 10 🌟: daily_briefing_tool         "What should I watch today?"
```

Plus `register_agent("planning", "PlanningAgent", tool_names=[...])`.

Integrated into `app/agents/tools.py` as side-effect import.

4. Validation

4.1 15 Structural Invariant Tests (5 categories)

Category	Tests	What's validated
1. Registration	all_10_tools, hard_write_permission, read_tier, agent_exists	LLM can invoke
2. Graceful errors	missing_part / product / part-no-bom	No raise, returns error dict
3. Three-segment	full_output, briefing_runs_empty, pricing_scenarios	summary + raw + warning
4. Slots	required_slots correctness	LLM can extract args
5. ConfirmCard	hard_write produces card, validates_input	Hard-write must use card

4.2 Results

15/15 tests pass. Sprint cumulative: 457/457 smoke tests pass.

5. Limitations and Future Work

Topic	Why not	Future direction
TTS voice output	Owner driving can't see screen	v3.31: Edge TTS / cloud TTS API
Whisper STT	Owner doesn't like typing	v3.31: Whisper integration (in Phase 4 roadmap)
OCR for quotes	Owner photos supplier quotes manually	v3.32: Tesseract + LLM field extraction
Email parsing	Procurement manually copies from email	v3.32: IMAP fetch + LLM extract
Mobile photo inventory	Warehouse staff manual entry	Phase 4 roadmap
Hierarchical query aggregation	"Show me this month's company-wide gross margin" needs LLM to auto-join queries	v3.31: multi-tool chaining auto-planning
Long-term memory	LLM doesn't remember "last time you asked"	v3.31: vector store personal history
Multi-language auto-detect	Currently zh-first	v3.31: auto-detect EN/ZH and switch response

6. Legal Notice / 法律聲明

⚠️ Important: Legal Nature of LLM Wrapper Layer

This module wraps **deterministic IE/OR algorithms** from v3.25.9 → v3.29 as LLM-callable tools. Customers must understand:

1. LLM Slot Extraction Error Risk

When owner says "Order 100 M6 bolts from ChangJiang at \$5 each," LLM may:

- Extract wrong product/part (e.g., "M6" matched to wrong spec)
- Extract wrong quantity ("100" could mean "100 sets = 500 pieces")
- Extract wrong unit ("5" — unit_cost or total?)

- **Miss critical fields** (delivery date, shipping address)

Therefore **all hard-writes must use ConfirmCard**, with human review of specific slot values before execution. This module's `commit_forecast_to_mps_with_confirm` strictly enforces this.

2. LLM Rendering Hallucination Risk

Read tools return `{summary, raw, warning}`. The `summary` is LLM-translated natural language. LLM may:

- **Over-simplify**: lose important nuance from raw data
- **Fabricate business reasons** (e.g., "Customer X cancelled" — without actual data)
- **Semantic drift**: interpret raw numbers with different meaning

Customers should view `raw` structured data for critical decisions, not solely trust LLM's `summary`.

3. Daily Briefing Aggregation Risk

`daily_briefing_tool` aggregates outputs from multiple downstream tools. Aggregation doesn't reduce risk — may amplify it:

- Each downstream tool's limitations **apply cumulatively** (see each tool's warning)
- Ordering / filtering / top-5 selection may **miss important events**
- If owner only reads briefing without drilling down, may **miss details**

Recommendation: click into each briefing item to inspect raw data.

4. Cumulative Applicability of Predecessor Disclaimers

This version overlays v3.25.10 → v3.29. **All predecessors' §6 disclaimers apply cumulatively**:

- v3.25.10 §6: MRP is planning suggestion, not PO
- v3.26 §6: CLSP heuristic non-optimal; capacity input accuracy responsibility
- v3.27 §6: Provenance ≠ legal causation; TOC heuristic; OAT doesn't catch interactions
- v3.28 §6: TA ≠ GAAP/IFRS; antitrust warning; DBR empirical
- v3.29 §6: Forecast not guarantee; unforeseen events; LLM business hallucination

5. RBAC × LLM Integration Compliance

Every LLM tool declares `required_permission` (e.g., `mps_mrp.master.create`). If user **lacks permission**:

- Tool refuses execution — LLM won't bypass
- Even if owner says "I'm admin," unverified speech is not trusted
- This is CONVERSATIONAL_ERP_DESIGN §5 principle #7: RBAC × AI integration cannot be skipped

6. Disclaimer Clause

To the maximum extent permitted by applicable law, erpilot assumes no liability for:

- **Consequences** of wrongly issued POs/orders due to LLM slot extraction errors (mitigated by ConfirmCard, but human button-press still possible)
- **Wrong business judgments** influenced by LLM translation inaccuracy
- **Missed important events** due to Daily Briefing omissions
- **False accusations against third parties** due to LLM-hallucinated business context
- **Major decision losses** from trusting **summary** without reviewing **raw**

7. Recommended Practice

- Large orders (> 5% of annual revenue) **must use UI forms with supervisor approval**, not just conversation
- Daily Briefing doesn't replace daily production meetings; **morning briefing aid only**
- Audit LLM interaction logs periodically; find common slot-extraction errors and update glossary
- Treat each AI query as "**consulting an expert**" not "**issuing a command**" — final authority remains human
- For outputs involving legal / antitrust / financial reporting (e.g., pricing curve), have **legal counsel / CPA review**

8. Cultural Reminder: LLM Doesn't Replace Professionals

erpilot's promise is "natural language replaces training" — not "natural language replaces professional judgment." AI can help owners operate quickly, find data, and do rough calculations; but final decisions still rest with:

- Sales / Procurement / Warehouse / Plant Manager — per their expertise
- Supervisor / Finance / Legal — per their authority
- Owner — per their role

Together.

7. References

- [1] Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv:1702.08608*.
- [2] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NIPS 30*.

[3] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the predictions of any classifier. *KDD '16*.

[4] Miller, G. A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2), 81-97.

[5] Schank, R. C. (1972). Conceptual dependency. *Cognitive Psychology*, 3(4), 552-631.

[6] Allen, J. F. (1995). *Natural Language Understanding* (2nd ed.). Benjamin/Cummings.

[7] Brown, T. et al. (2020). Language Models are Few-Shot Learners. *NeurIPS 33*.

[8] Schick, T., et al. (2023). Toolformer: Language models can teach themselves to use tools. *NeurIPS 36*.

[9] Yao, S., et al. (2022). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023*.

[10] erpilot **CONVERSATIONAL_ERP_DESIGN** (2026, internal). 6-layer architecture + 7 design principles + 4-phase roadmap.

[11-50] All references from v3.25.9 → v3.29 design docs **apply cumulatively**.

8. Changelog

Version	Date	Change
v3.25.9 - v3.29	2026-05-20	Algorithm foundation (IE/OR / TA / Forecasting)
v3.30	2026-05-20	This release: wrap all v3.25.9-v3.29 algorithms as LLM tools , fulfilling erpilot's North Star
Future v3.31+	TBD	TTS / Whisper / multi-tool chaining / vector memory
Future v3.32+	TBD	OCR quotes / email parsing / mobile photo inventory

Last updated: 2026-05-20 (v3.30) **Authors:** erpilot engineering team (with NLU / XAI / IE-OR cross-domain academic methodology) **Version:** 1.0 **Chinese version:** [CONVERSATIONAL_PLANNING_DESIGN_ZH.md](#)